ОСОБЕННОСТИ ВНЕДРЕНИЯ СИСТЕМЫ АНТИ-ФРОД В ДВУХ РАЗНЫХ ПЛАТФОРМАХ АВТОМАТИЗАЦИИ GITLAB-CI И GITHUB ACTIONS - ОТЛИЧИЯ, ПРЕИМУЩЕСТВА И НЕДОСТАТКИ

ПРЕЗЕНТАЦИЯ ИТОГОВОГО ПРОЕКТА OTUS / *АКИМОВ ПЕТР ИГОРЕВИЧ*

ВНЕДРЕНИЕ АНТИ-ФРОДА GITLAB-CI VS. GHA

СТУДЕНТ: АКИМОВ ПЕТР ИГОРЕВИЧ

КУРС: MLOPS

ДАТА: 15.04.2026

О ПРОЕКТЕ

- Цель проекта: продемонстрировать особенности популярных CI-систем в разрезе внедрения системы анти-фрод
- Целевая аудитория: MLOps-инженеры
- Особенности проекта:

  - выполнено на публичной облачной платформе Яндекс Облако

  - алгоритм машинного обучения предложен автоматически с помощью AutoML инструментов

  - рассмотрены популярные CI-системы: GitHub Actions vs. GitLab-CI

  - реализован инференс на потоке с настройкой сдвига данных и TPS

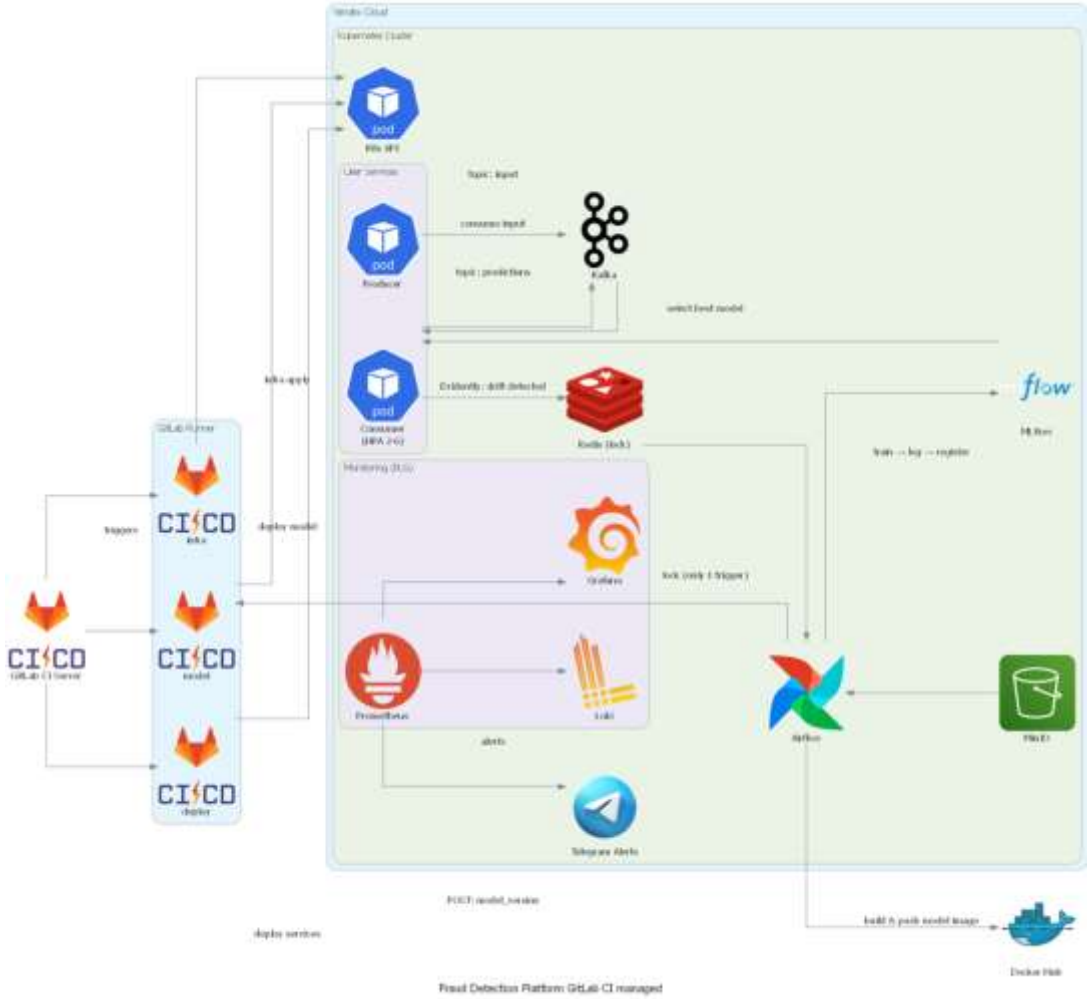
ТЕХНОЛОГИИ И ИНСТРУМЕНТЫ

1. Python
2. FLAML
3. XGBoost
4. Apache Kafka
5. MLflow
6. Apache Airflow
7. GitHub Actions
8. GitLab-CI
9. Kubernetes
10. Evidently

ЭТАПЫ РАЗРАБОТКИ

1. Подготовка данных
2. Подбор модели
3. Внедрение модели

ДЕМОНСТРАЦИЯ ПРОЕКТА



Ссылки на проекты https://gitlab.com/otus5491242/final
https://github.com/petr-akimov/otus-final

ТРУДНОСТИ И РЕШЕНИЯ

1. Реплик пода с моделью много, а POST-запрос к API CI-системы не должен дублироваться

   - Решение: использовать Redis для блокировки POST-запросов

2. Как эмулировать сдвиг данных в рамках учебного проекта?

   - Решение: подготовить два датасета таким образом, чтобы Evidently обнаружил сдвиг

3. Алгоритмов машинного обучения много, времени мало, и присутствует человеческий фактор

   - Решение: делегировать право выбора алгоритма машинного обучения самим машинам

4. Если многие алгоритмы дают приблизительно один и тот же ROC-AUC > 99%, то какую метрику сделать решающей?

   - Решение: комплексная метрика (ROC-AUC > 99% + min_inference_time)

ОСОБЕННОСТИ GHA И GITLAB В РАЗРЕЗЕ ВНЕДРЕНИЯ СИСТЕМ АНТИ-ФРОД

Параметр	GitLab	GitHub Actions	Вывод для финтеха
Контроль данных и on-prem	Полный self-hosted, air-gap, контроль хранения и логов	Ограничен (только Enterprise Server, сложнее изоляция)	**GitLab предпочтительнее**
Управление секретами	Нативная интеграция с HashiCorp Vault, JWT, zero-trust подход	Secrets + внешние плагины	**GitLab безопаснее**
Встроенные DevSecOps функции (единый параметр)	Из коробки: registry, SAST/DAST, dependency scanning, container scanning, MLflow-compatible tracking, compliance pipelines	Через marketplace + платные add-ons	**GitLab снижает риски**
Supply chain risk (зависимость от стороннего кода)	Низкий (меньше внешних зависимостей)	Высокий (actions из marketplace)	**GitLab менее уязвим**
Audit / Compliance	Централизованные audit logs, policy-as-code, проще проходить PCI DSS / ISO	Есть, но менее централизовано	**GitLab лучше для регуляторов**
Изоляция CI execution (runners)	Полный контроль (private runners, внутри периметра)	Shared runners + сложнее контроль self-hosted	**GitLab безопаснее**
Vendor lock-in	Средний	Высокий (экосистема GitHub)	**GitLab гибче**
CAPEX (on-prem запуск)	Средний (инфра + лицензия)	Высокий (Enterprise + доп. сервисы)	**GitLab выгоднее**
OPEX (поддержка)	Средний (единая платформа)	Выше (много внешних сервисов)	**GitLab дешевле в долгосрок**
Стоимость старта (time-to-market)	Средняя	Низкая (быстрый старт)	**GitHub быстрее**
Требования к команде	Нужен сильный DevSecOps	Ниже порог входа	**GitHub проще**
Масштабирование и рост	Лучше для сложных enterprise систем	Хорошо, но через интеграции	**GitLab стабильнее**
Подход к MLOps (anti-fraud)	Частично встроено (MLflow API + registry)	Всё отдельно	**GitLab проще и безопаснее**
Общая поверхность атаки	Меньше (монолитная платформа)	Больше (ecosystem + плагины)	**GitLab менее рискован**

БЛАГОДАРНОСТИ

- Руководителю курса и преподавателю, проверявшему домашние задания
- Авторам курса, лекторам на вебинарах
- Всему персоналу OTUS, причастному к курсу, его организации, поддержке

ВАШИ ВОПРОСЫ?

СПАСИБО

Петр Акимов

@petrakimov